

Angular Note 4.1: Directive Fundamentals

Module: Directives - Structural and Attribute

Topic: Understanding Directives in Angular

Duration: 2-3 hours

Introduction

Angular directives are one of the most powerful features of the framework. They extend HTML with custom behavior, allowing developers to manipulate the DOM, add functionality to elements, and create reusable UI logic. Understanding directives is fundamental to building dynamic and interactive Angular applications.

This note covers the conceptual foundation of directives, their relationship with components, and the different types available in Angular.

1. What are Directives?

1.1 Definition

A **directive** is a class with a `@Directive` decorator that adds custom behavior to elements in Angular applications. Directives allow you to:

- Manipulate the DOM structure (add or remove elements)
- Change the appearance or behavior of elements
- Extend HTML with custom attributes
- Create reusable UI logic across components

Key Concept: Directives teach new tricks to HTML elements. They tell Angular how to transform the DOM when rendering the template.

1.2 The Purpose of Directives

Directives serve several critical purposes in Angular applications:

Purpose	Description	Example
DOM Manipulation	Add, remove, or modify DOM elements	<code>*ngIf</code> removes elements
Behavioral Enhancement	Add event listeners or interaction logic	Click handlers, hover effects
Styling Control	Dynamically apply CSS classes or styles	<code>ngClass</code> , <code>ngStyle</code>
Code Reusability	Share common functionality across components	Custom validation directives

1.3 How Directives Extend HTML

Standard HTML has limited built-in functionality. Directives extend HTML by adding custom attributes that Angular recognizes and processes.

Example: Built-in HTML vs Angular Directives

```
<!-- Standard HTML - Limited functionality -->
<div class="box">Always visible</div>

<!-- Angular with Directive - Enhanced functionality -->
<div *ngIf="isVisible" class="box">Conditionally visible</div>
```

In the second example, `*ngIf` is a directive that adds conditional rendering capability to the standard `<div>` element.

2. Directives vs Components

2.1 The Relationship

Here's an important fact: **Components are actually directives with templates**. A component is a special type of directive that includes a view (HTML template).

```
@Component({
  selector: 'app-user',
  template: '<h1>User Component</h1>'
})
export class UserComponent { }

// Component = Directive + Template
```

2.2 Key Differences

Aspect	Directive	Component
Decorator	@Directive	@Component
Template	No template	Has template/templateUrl
Purpose	Add behavior to existing elements	Create new UI components with view
Usage	Applied as attribute to host element	Used as custom element
View Encapsulation	Not applicable	Has view encapsulation
Quantity per Element	Multiple directives on one element	One component per element

2.3 When to Use Each

Use a Component when:

- You need to create a new UI element with its own view

- Building a reusable widget or feature
- Creating a page or section with complex layout
- Need to manage state and lifecycle for a specific view

Use a Directive when:

- You want to add behavior to existing HTML elements
- Modifying appearance or attributes of elements
- Creating reusable functionality without a template
- Manipulating DOM structure conditionally

2.4 Practical Comparison Example

Scenario 1: Creating a User Profile Card (Use Component)

```
// Component with template
@Component({
  selector: 'app-user-card',
  template: `
    <div class="card">
      <img [src]="user.avatar">
      <h3>{{ user.name }}</h3>
      <p>{{ user.email }}</p>
    </div>
  `
})
export class UserCardComponent {
  @Input() user: any;
}

// Usage
<app-user-card [user]="currentUser"></app-user-card>
```

Scenario 2: Adding Highlight on Hover (Use Directive)

```
// Directive without template
@Directive({
  selector: '[appHighlight]'
})
```

```
export class HighlightDirective {
  @HostListener('mouseenter') onMouseEnter() {
    // Add behavior to existing element
    this.el.nativeElement.style.backgroundColor = 'yellow';
  }

  constructor(private el: ElementRef) {}
}

// Usage - Applied to any existing element
<p appHighlight>Hover over me!</p>
<div appHighlight>This also works!</div>
```

Output:

Conceptual Understanding:

Component: Creates a complete new element with structure (user card with image, name, email)

Directive: Enhances existing elements (any paragraph or div can get hover highlighting)

3. Types of Directives

Angular provides three types of directives, each serving a specific purpose:

3.1 Component Directives

Component directives are directives with templates. They create custom HTML elements.

```
@Component({
  selector: 'app-button',
  template: '<button>{{ label }}</button>'
})
export class ButtonComponent {
  @Input() label: string = 'Click Me';
}
```

```
// Usage
<app-button label="Submit"></app-button>
```

Characteristics:

- Most commonly used directive type
- Has associated template and styles
- Creates reusable UI blocks
- Can contain other components and directives

3.2 Structural Directives

Structural directives change the DOM layout by adding, removing, or manipulating elements. They are identified by the asterisk (*) prefix.

```
<!-- *ngIf adds or removes element from DOM -->
<div *ngIf="isLoggedIn">
  Welcome back!
</div>

<!-- *ngFor repeats element for each item -->
<ul>
  <li *ngFor="let item of items">{{ item }}</li>
</ul>

<!-- *ngSwitch creates conditional sections -->
<div [ngSwitch]="userRole">
  <p *ngSwitchCase="'admin'">Admin Panel</p>
  <p *ngSwitchCase="'user'">User Dashboard</p>
  <p *ngSwitchDefault>Guest View</p>
</div>
```

Characteristics:

- Prefixed with asterisk (*)
- Modify the structure of the DOM
- Work with `<ng-template>` internally
- Only one structural directive per element

Important Rule: You cannot apply two structural directives to the same element. This will cause an error:

```
<!-- ✗ WRONG - Multiple structural directives -->
<div *ngIf="isVisible" *ngFor="let item of items"></div>

<!-- ✓ CORRECT - Use container element -->
<div *ngIf="isVisible">
  <div *ngFor="let item of items"></div>
</div>
```

3.3 Attribute Directives

Attribute directives change the appearance or behavior of an element, component, or another directive.

```
<!-- ngClass dynamically adds/removes CSS classes -->
<div [ngClass]="{'active': isActive, 'disabled': !isEnabled}">
  Dynamic Classes
</div>

<!-- ngStyle dynamically sets inline styles -->
<div [ngStyle]="{'color': textColor, 'font-size': fontSize + 'px'}">
  Dynamic Styles
</div>

<!-- Custom attribute directive -->
<p appHighlight highlightColor="yellow">
  Highlighted text
</p>
```

Characteristics:

- Change element appearance or behavior
- No asterisk prefix
- Applied as attributes
- Multiple attribute directives can be used on one element
- Don't change DOM structure

3.4 Directive Types Comparison Table

Feature	Component	Structural	Attribute
Has Template	☒ Yes	☒ No	☒ No
Modifies DOM Structure	☒ Yes (creates new)	☒ Yes (add/remove)	☒ No
Modifies Appearance	☒ Yes	☒ No	☒ Yes
Prefix	None (custom tag)	*	None
Multiple on Element	☒ No	☒ No	☒ Yes
Examples	<code><app-header></code>	<code>*ngIf</code> , <code>*ngFor</code>	<code>ngClass</code> , <code>ngStyle</code>

4. Comprehensive Example: All Three Types Together

Let's see how all three directive types work together in a real application.

Component TypeScript (app.component.ts)

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  // Data for demonstration
  isLoggedIn = true;
  userRole = 'admin';

  products = [
    { id: 1, name: 'Laptop', price: 999, inStock: true },
    { id: 2, name: 'Mouse', price: 25, inStock: true },
    { id: 3, name: 'Keyboard', price: 75, inStock: false },
    { id: 4, name: 'Monitor', price: 299, inStock: true }
  ]
}
```

```
];

toggleLogin() {
  this.isLoggedIn = !this.isLoggedIn;
}

changeRole(role: string) {
  this.userRole = role;
}
}
```

Component Template (app.component.html)

```
<div style="padding: 20px;">
  <h2>Angular Directives in Action</h2>

  <!-- Control buttons -->
  <button (click)="toggleLogin()">
    {{ isLoggedIn ? 'Logout' : 'Login' }}
  </button>

  <button (click)="changeRole('admin')">Admin</button>
  <button (click)="changeRole('user')">User</button>
  <button (click)="changeRole('guest')">Guest</button>

  <hr>

  <!-- STRUCTURAL DIRECTIVE: *ngIf -->
  <div *ngIf="isLoggedIn" style="padding: 10px; background-color: #e8f5e9; border-radius:
5px;">
    <h3><input checked="" type="checkbox"/> You are logged in</h3>
  </div>

  <div *ngIf="!isLoggedIn" style="padding: 10px; background-color: #ffebee; border-radius:
5px;">
    <h3><input type="checkbox"/> Please log in to continue</h3>
  </div>

  <hr>

  <!-- STRUCTURAL DIRECTIVE: *ngSwitch -->
  <div [ngSwitch]="userRole">
    <div *ngSwitchCase="'admin'"
      style="padding: 15px; background-color: #fff3cd; border-left: 4px solid #ffc107;">
```

```

    <h3>🔑 Admin Dashboard</h3>
    <p>Full access to all features</p>
  </div>

  <div *ngSwitchCase="'user'"
    style="padding: 15px; background-color: #d1ecf1; border-left: 4px solid #17a2b8;">
    <h3>👤 User Dashboard</h3>
    <p>Standard user privileges</p>
  </div>

  <div *ngSwitchDefault
    style="padding: 15px; background-color: #e2e3e5; border-left: 4px solid #6c757d;">
    <h3>🕒 Guest View</h3>
    <p>Limited access - please register</p>
  </div>
</div>

<hr>

<!-- STRUCTURAL DIRECTIVE: *ngFor with ATTRIBUTE DIRECTIVES: ngClass, ngStyle -->
<h3>Product Catalog</h3>
<div *ngFor="let product of products; let i = index; let isFirst = first; let isLast = last"
  [ngClass]='{"first-item": isFirst, "last-item": isLast, "out-of-stock": !
product.inStock}'
  [ngStyle]="{'border-left': product.inStock ? '4px solid #4caf50' : '4px solid #f44336'}"
  style="padding: 10px; margin: 10px 0; background-color: #f5f5f5; border-radius: 5px;">

  <strong>{{ i + 1 }}. {{ product.name }}</strong> -
  <span [ngStyle]="{'color': product.inStock ? 'green' : 'red', 'font-weight': 'bold'}">
    ${{ product.price }}
  </span>

  <span *ngIf="product.inStock"
    style="margin-left: 10px; color: green;">
    ☒ In Stock
  </span>

  <span *ngIf="!product.inStock"
    style="margin-left: 10px; color: red;">
    ☒ Out of Stock
  </span>
</div>
</div>

```

Component CSS (app.component.css)

```
.first-item {  
  border-top: 3px solid #2196f3;  
  font-weight: bold;  
}  
  
.last-item {  
  border-bottom: 3px solid #2196f3;  
}  
  
.out-of-stock {  
  opacity: 0.6;  
  background-color: #ffebee !important;  
}  
  
button {  
  margin: 5px;  
  padding: 10px 15px;  
  background-color: #4CAF50;  
  color: white;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
}  
  
button:hover {  
  background-color: #45a049;  
}
```

Output:

Browser Display:

Initial State (Logged In as Admin):

1. Four buttons appear: "Logout", "Admin", "User", "Guest"
2. A green box shows "☒ You are logged in"
3. A yellow box with key icon shows " Admin Dashboard - Full access to all features"
4. Product list shows:
 - 1. Laptop - \$999 ☒ In Stock (green border, bold top border as first item)

- 2. Mouse - \$25 ☒ In Stock (green border)
- 3. Keyboard - \$75 ☒ Out of Stock (red border, faded background)
- 4. Monitor - \$299 ☒ In Stock (green border, bold bottom border as last item)

After Clicking "Logout":

- Button changes to "Login"
- Green box disappears, red box appears: "☒ Please log in to continue"
- Product list remains visible

After Clicking "User":

- Yellow admin panel disappears
- Blue box appears: "👤 User Dashboard - Standard user privileges"

After Clicking "Guest":

- Gray box appears: "👁️ Guest View - Limited access - please register"

Directives in Action:

- `*ngIf` shows/hides login status
- `*ngSwitch` displays different dashboard based on role
- `*ngFor` repeats product items
- `ngClass` applies first-item, last-item, out-of-stock classes
- `ngStyle` sets dynamic border colors and text colors

5. Directive Selector Patterns

Directives can use different CSS selector patterns:

5.1 Element Selector (Components Only)

```
@Component({  
  selector: 'app-user'  
})  
// Usage: <app-user></app-user>
```

5.2 Attribute Selector (Most Common for Directives)

```
@Directive({  
  selector: '[appHighlight]'  
})  
// Usage: <p appHighlight>Text</p>
```

5.3 Class Selector

```
@Directive({  
  selector: '.app-highlight'  
})  
// Usage: <p class="app-highlight">Text</p>
```

5.4 Attribute with Value

```
@Directive({  
  selector: '[appHighlight][color]'  
})  
// Usage: <p appHighlight color="yellow">Text</p>
```

6. Best Practices

6.1 Naming Conventions

- Use `app` prefix for custom directives: `appHighlight`, `appTooltip`

- Use descriptive names that indicate purpose: `appClickOutside` , `appDragDrop`
- Follow camelCase for directive selectors

6.2 When to Create Custom Directives

- When you need reusable behavior across multiple components
- For DOM manipulations that don't require a template
- To encapsulate complex event handling logic
- For form validation that applies to multiple inputs

6.3 Performance Considerations

- Structural directives create/destroy DOM nodes - use wisely
- Minimize use of structural directives in loops
- Prefer `ngClass` over multiple class bindings
- Avoid complex expressions in directive inputs

Summary

Directives are instructions to Angular on how to transform the DOM:

1. What are Directives?

- Classes with `@Directive` decorator
- Extend HTML with custom behavior
- Manipulate DOM and add functionality

2. Directives vs Components:

- Components are directives with templates
- Use components for new UI elements with views
- Use directives for enhancing existing elements

3. Three Types:

- **Component Directives:** Create new elements with templates
- **Structural Directives:** Change DOM layout (`*ngIf` , `*ngFor` , `*ngSwitch`)
- **Attribute Directives:** Change appearance/behavior (`ngClass` , `ngStyle` , custom)

Understanding these fundamentals prepares you for creating powerful, reusable functionality in Angular applications. The next notes will dive deeper into structural directives, attribute directives, and creating custom directives.

End of Angular Note 4.1: Directive Fundamentals

